

Content

OpenVPN on AWS: Secure Access to Private Infrastructure

Introduction

Many DevOps engineers and Site Reliability Engineers (SREs) have likely worked on projects where the highest priority during the early stages was always:

┆ "Get the system running first."

When deadlines are tight, infrastructure must be provisioned quickly, and environments need to be ready immediately for developers to deploy applications, many trade-offs are often made in exchange for deployment speed. SSH is exposed publicly for easier access. Grafana, Jenkins, or ArgoCD are directly accessible from the Internet for easier troubleshooting. Sometimes even databases are temporarily opened to public access simply so the team can move faster.

At that moment, everyone knows:

┆ "This isn't ideal."

But most people also think:

┆ ""It can be improved later."

Then real users start arriving, traffic increases, and the production environment begins operating at scale. And that is exactly when security debt starts demanding repayment.

- Is it really safe when every service is exposed to the Internet?
- How can engineers still access private resources without exposing the entire system?
- How do we maintain operational convenience without turning production into a massive attack surface?

In this article, we will explore:

- What VPN actually is
- Why VPN remains extremely important
- How OpenVPN works
- A hands-on demonstration of deploying OpenVPN Access Server on AWS and securely accessing private resources.

This article shares practical lessons learned from operating real-world infrastructure.

Section 1 — VPN Fundamentals

1. What Is a VPN?

VPN (Virtual Private Network) is a technology that creates a private and encrypted connection over the public Internet infrastructure.

Instead of exposing the entire system directly to the Internet for access, we create a secure tunnel between the user's computer and the internal system.

Once the VPN connection is established successfully, your device behaves almost as if it were located inside the company's private network or cloud infrastructure.

That is why, after enabling VPN, you can:

- SSH into private EC2 instances
- Access internal databases
- Open non-public Grafana dashboards
- Interact with private Kubernetes API endpoints

From the system's perspective, your laptop essentially becomes "a member" of the internal network.

2. What Problems Does VPN Solve?

One common misconception is that VPN is only used to change IP addresses, hide locations and stay anonymous. In reality, those are merely side effects. The true value of VPN lies in creating a secure connection between users and internal systems.

Accessing Private Resources from the Internet

? If everything is publicly exposed to the Internet, access certainly becomes convenient — but it also means anyone on the Internet can “knock on the door.”

In properly designed systems, many resources are intentionally kept private, such as: EC2 instances, databases, Kubernetes control planes, Jenkins servers. However, operations teams still need daily access to them.

- Developers need SSH access for debugging
- DevOps engineers need monitoring access
- SREs need to handle midnight incidents

✓ VPN solves this problem by creating a secure connection between the user device and the private network. After connecting to the VPN, users can access private resources as if they were operating inside the same internal network.

Reducing the Attack Surface

? Every service exposed to the Internet — such as SSH, RDP, PostgreSQL, or administrative dashboards — can become a target for port scanning, brute-force attacks, vulnerability exploitation. As systems grow larger, temporarily exposing “just a few services” often evolves into dozens of security groups, numerous exceptions, complex rule chains. Eventually, the environment becomes difficult to manage securely.

✓ VPN reverses this mindset. Instead of exposing services and attempting to protect them afterward, we keep services private from the beginning. Only users with valid accounts, successful authentication, authorized VPN access can reach internal resources.

Replacing Traditional IP Whitelisting & Protecting Data on Public Networks

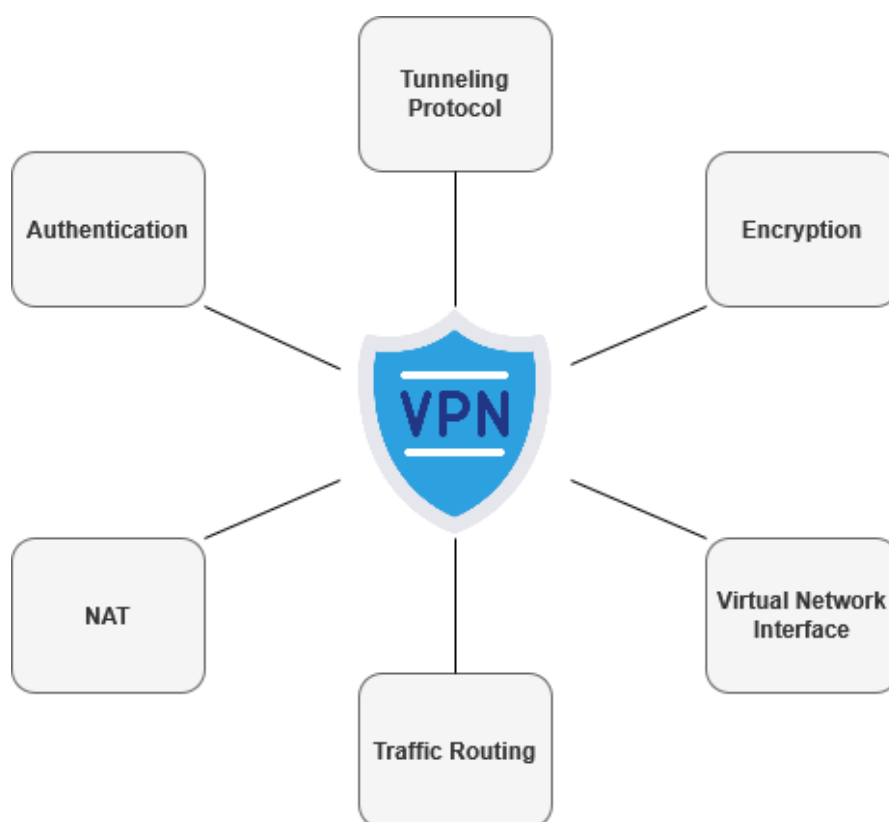
? IP whitelisting is a common approach for restricting access by allowing only specific IP addresses. This works well... Until engineers work remotely, travel for business, use mobile networks, sit in coffee shops. Their IP addresses constantly change, leading to requests such as “Can you whitelist my new IP”, “My Wi-Fi changed”, “I’m at the airport”, “I urgently need production access”.

✓ VPN approaches the problem differently. Instead of trusting IP addresses, we trust user identities, certificates, authentication mechanisms, multi-factor authentication (MFA). As long as authentication succeeds, users can securely access the system from anywhere. VPN also creates an encrypted

communication channel between the user device and the VPN server, protecting data during transmission.

3. How Does VPN Work?

Behind the seemingly simple “Connect” button, many components work together.



- **Authentication:** Users must authenticate with the VPN server using username/password, certificates, MFA. Only authorized users can establish VPN connections
- **Tunneling Protocol & Encryption:** After authentication succeeds, the VPN establishes an encrypted tunnel between the VPN client and VPN server. All traffic transmitted through this tunnel is encrypted. As a result VPN significantly reduces the risk of traffic sniffing when using public Wi-Fi. Common protocols include TLS, SSL, IPSec
- **Virtual Network Interface:** Once the VPN connection is established, the operating system creates a virtual network interface. For example: Windows creates a new adapter, Linux creates `tun0` or `tap0`, macOS creates a similar interface. At the same time, the device receives an IP

address belonging to the VPN network. At that moment, your laptop officially becomes part of the VPN network

- **Traffic Routing / Route Table:** The VPN client receives required routes from the VPN server. As a result, traffic destined for private networks goes through the VPN tunnel, normal Internet traffic may still go directly to the Internet depending on configuration. This is exactly how Split Tunnel works
- **NAT:** In many VPN deployment models, the VPN server performs NAT or routing to allow VPN clients to communicate with the private network behind it. Depending on the architecture, VPN clients may be routed directly, or NATed through the VPN server before accessing internal resources

4. Full Tunnel vs Split Tunnel

After a VPN connection is established, there are two common routing models.

Criteria	Full Tunnel	Split Tunnel
Traffic through VPN	All traffic	Only internal traffic
VPN Server bandwidth usage	High	Lower
Internet performance	Lower	Better
Level of control	High	Moderate
Suitable for	Enterprises needing full traffic control	DevOps, SRE, Cloud Infrastructure

Throughout this guide, we will use Split Tunnel to access private resources on AWS while allowing normal Internet traffic to go directly to the Internet.

Now that we understand the fundamentals of VPN technology, let's explore why OpenVPN remains one of the most widely adopted VPN solutions today.

Section 2 — OpenVPN Solution Overview

1. Popular VPN Solutions

Today, many VPN solutions are available on the market. Each solution is designed for different use cases and environments.

Solution	Highlights	Suitable For
OpenVPN	Open-source, stable, widely adopted	Personal use, enterprise, cloud
WireGuard	High performance, modern, simple configuration	Cloud, Kubernetes, homelab
IPSec VPN	Traditional VPN standard integrated into networking devices	Enterprise networks
AWS Client VPN	AWS-managed VPN service	Pure AWS environments
Site-to-Site VPN	Connects two separate networks	Hybrid cloud, data centers

In this article, we choose OpenVPN because it is almost a “classic” technology in the infrastructure world:

- Beginner-friendly
- Large community support
- Rich documentation
- Multi-platform support
- Excellent for understanding VPN fundamentals

Additionally, many modern VPN solutions still inherit concepts similar to OpenVPN, so understanding OpenVPN also helps explain how other VPN systems operate.

2. OpenVPN Community Edition vs Access Server

OpenVPN currently has two common deployment approaches:

- OpenVPN Community Edition
- OpenVPN Access Server

Although both are based on the same OpenVPN technology, they target different audiences.

Criteria	Community Edition	Access Server
Cost	Completely free	Free with limited concurrent connections
Management Interface	CLI	Web UI

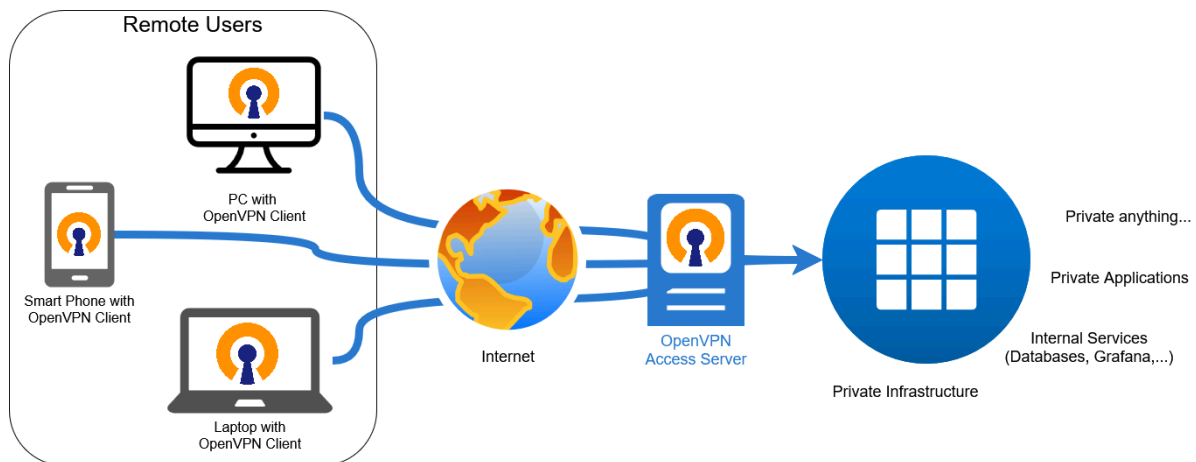
Criteria	Community Edition	Access Server
User Management	Manual	Built-in
Certificate Management	Manual	Automated
Routing Configuration	Manual	Web UI
MFA Support	Manual setup	Built-in
Deployment Complexity	High	Low
Primary Use Case	Learning, deep customization	Fast deployment, easy operations

In this article, instead of spending too much time on manual configuration, we will use Access Server to focus more on VPN concepts, networking flow and operational workflows.

3. OpenVPN Access Server Architecture

One reason OpenVPN Access Server is widely adopted is because its architecture is relatively simple and easy to deploy.

In a basic deployment model, the Access Server acts as the gateway into the internal network.



When users connect to the VPN:

1. OpenVPN Access Server authenticates the user
2. A VPN tunnel is established
3. An IP address belonging to the VPN network is assigned

4. Required routes are pushed to the VPN client
5. Traffic is forwarded to internal resources

With the core concepts covered, we can now move on to deploying a practical OpenVPN environment on AWS.

Section 3 — Deploying OpenVPN Access Server on AWS

1. Demo Objective — Hands-On Time

At this stage, the theory should provide a solid understanding of what VPN is, how it works and why it appears in nearly every modern cloud system. So instead of continuing with concepts, we will build a real AWS lab environment to observe how VPN traffic behaves in a production-like environment.

The objectives of this demo are straightforward:

- Connect a personal laptop to AWS through VPN
- Access an EC2 instance located inside a private subnet
- SSH into an instance without a public IP
- Observe how traffic is routed through the VPN tunnel

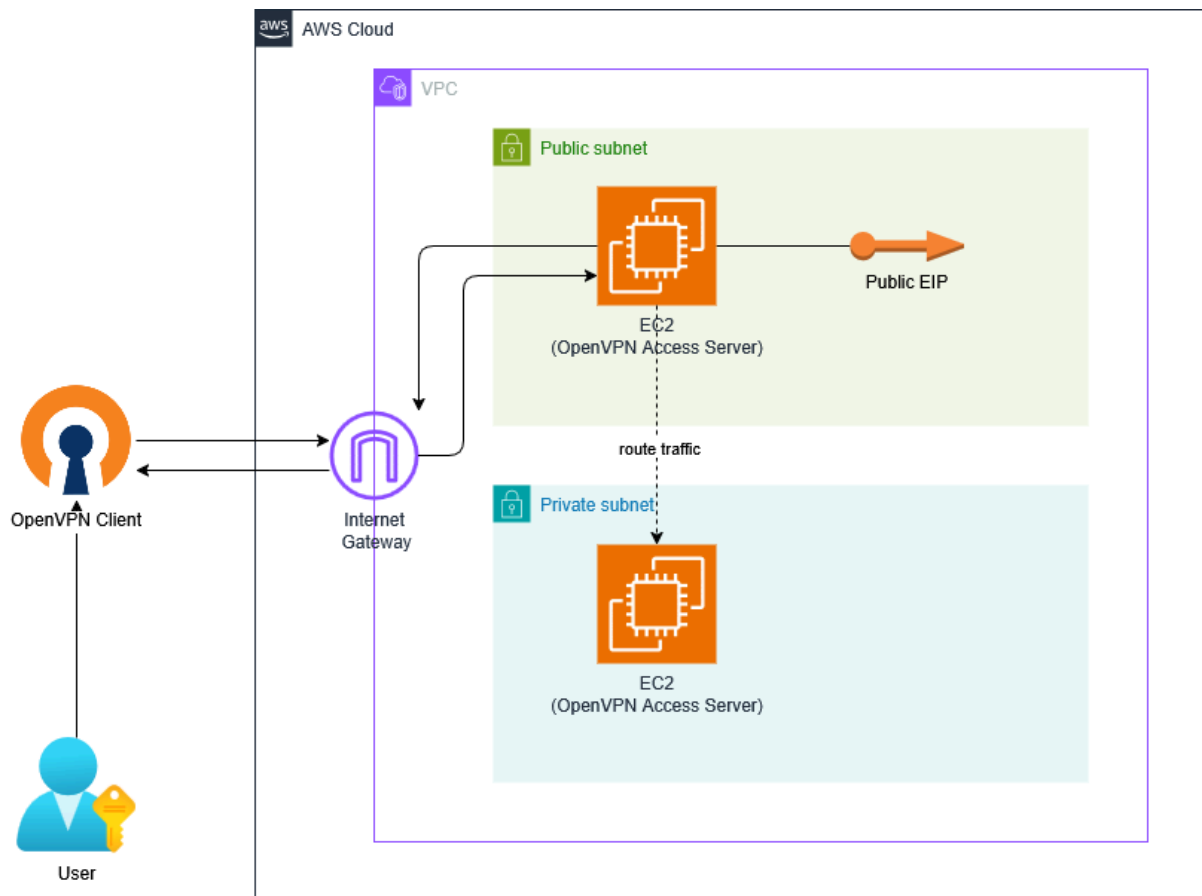
If everything works correctly, our laptop connected from the public Internet will be able to access private infrastructure as if it were located inside the same internal network.

That is the real value of VPN in cloud environments.

2. AWS Architecture

To focus on VPN and networking flow instead of complicated infrastructure, we will use a minimal architecture consisting of:

- 1 VPC
- 1 Public Subnet
- 1 Private Subnet
- 1 OpenVPN Access Server
- 1 Private EC2 Instance



In this architecture:

- OpenVPN Access Server accepts VPN connections from the Internet
- The private EC2 instance has no public IP
- All access to the private EC2 instance must go through the VPN

This architecture reflects how many organizations secure production environments on AWS.

3. Preparing AWS Infrastructure

The goal of this article is to focus on VPN and networking concepts rather than detailed infrastructure provisioning. Therefore, instead of manually creating every AWS resource, we will use a prepared Terraform configuration to provision the lab environment quickly.

[raw.githubusercontent.com](https://raw.githubusercontent.com/orez-fu/blog_content/refs/heads/master/vpn/part_1/main.tf)

https://raw.githubusercontent.com/orez-fu/blog_content/refs/heads/master/vpn/part_1/main.tf

Terraform provisions the following components:

Resource	Purpose
VPC	Private network
Public Subnet	Hosts OpenVPN Access Server
Private Subnet	Hosts Private EC2
Internet Gateway	Provides Internet access
Route Table	Controls traffic routing
Security Group	Controls network access
EC2 Instance	OpenVPN Server with Elastic IP
EC2 Instance	Private test server without Public IP
Elastic IP	Public IP for OpenVPN Server

Instead of installing OpenVPN manually by:

- Generating CA certificates
- Creating certificates
- Configuring daemons
- Setting up routing
- Configuring NAT
- Managing iptables

we will use the official OpenVPN Access Server AMI from AWS Marketplace. This is a common real-world approach because it enables faster deployment, easier maintenance, practical lab and PoC environments.

Before Terraform can provision the EC2 instance using this AMI, AWS requires the account to subscribe to the Marketplace product first.

Access the OpenVPN Access Server Marketplace product:

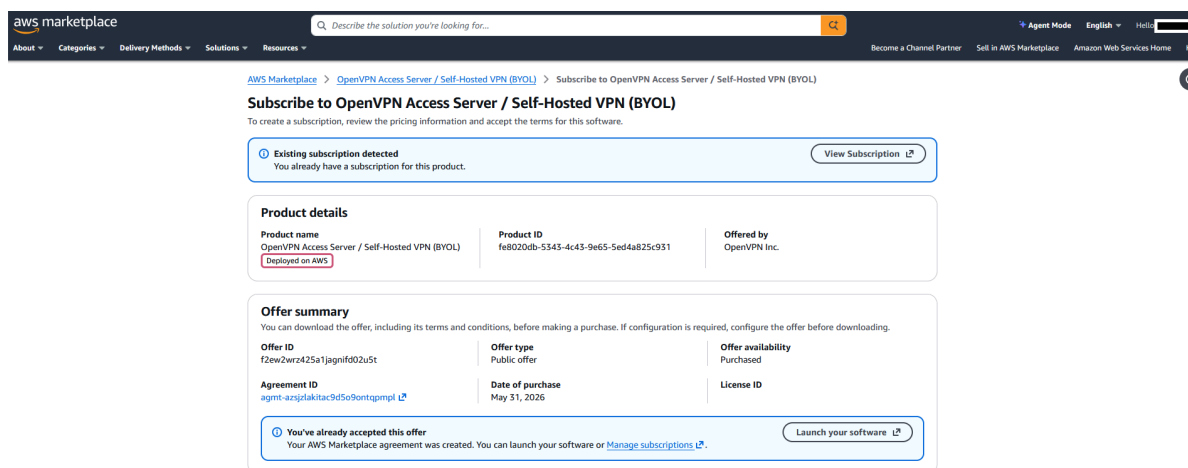
AWS Marketplace Procurement Page

 <https://aws.amazon.com/marketplace/procurement/?productId=fe8020db-5343-4c43-9e65-5ed4a825c931&redirectUrl=https%3A%2F%2Faws.amazon.com%2Fmarketplace%2Fpp%2Fprodview-y3m73u6jd5srk>

Then follow these steps:

- Select **Continue to Subscribe**
- Accept the terms and conditions (**Accept Terms**)
- Wait until the status changes to **Subscribed**

Once completed, you can monitor the subscription status. Please note that the OpenVPN Access Server version used in this article is the **BYOL (Bring Your Own License)** edition. This version allows free usage with a limited number of concurrent connections, making it suitable for learning and testing purposes.



After completing the Marketplace subscription, we can begin setting up the lab environment. Make sure your computer has:

- Install AWS CLI (<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>)
- Configure AWS credentials (<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-quickstart.html>)
- Install Terraform CLI: (<https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli>)

Next, download the Terraform source code and provision the AWS infrastructure using the following commands:

```
terraform init
terraform plan
```

```
terraform apply -auto-approve
```

The provisioning process may take several minutes.

Once completed, Terraform outputs the public IP address assigned to the OpenVPN Access Server.

```
aws_subnet.private: Creation complete after 2s [id=subnet-07b7c4d686678b3e0]
aws_route_table.public: Creation complete after 3s [id=rtb-0964e5d9d32ccc3ed]
aws_security_group.private_ec2: Creation complete after 6s [id=sg-09878e6e5759ca509]
aws_security_group.openvpn: Creation complete after 6s [id=sg-0f67f7716e1a87e51]
aws_instance.private_ec2: Creating...
aws_subnet.public: Still creating... [00m10s elapsed]
aws_instance.private_ec2: Still creating... [00m10s elapsed]
aws_subnet.public: Creation complete after 16s [id=subnet-06ebf7d972dc04cd3]
aws_route_table_association.public: Creating...
aws_instance.openvpn: Creating...
aws_route_table_association.public: Creation complete after 1s [id=rtbassoc-069208e3651bdd18f]
aws_instance.private_ec2: Creation complete after 16s [id=i-0c4d4158ddd6f2873]
aws_instance.openvpn: Still creating... [00m10s elapsed]
aws_instance.openvpn: Creation complete after 14s [id=i-0be82aae63ea0ef0d]
aws_eip_association.openvpn: Creating...
aws_eip_association.openvpn: Creation complete after 2s [id=eipassoc-0fc5dc300f5f3b7d6]

Apply complete! Resources: 15 added, 0 changed, 0 destroyed.

Outputs:

openvpn_public_ip = "100.50.141.150"
private_ec2_ip = "10.0.2.147"
private_ec2_ssh_command = "ssh -i ./vpn-demo-ssh-key.pem ec2-user@10.0.2.147"
ssh_key_pair_name = "vpn-demo-ssh-key"
ssh_private_key_file = "./vpn-demo-ssh-key.pem"
ssh_private_key_pem = <sensitive>
```

4. Installing OpenVPN Access Server

4.1 Creating an OpenVPN Access Server Account

Visit the following link: <https://myaccount.openvpn.com/signin/product-select> and register for a new account, or sign in using your Google or Microsoft account.



Secure your business network in minutes

Work Email *

name@domain.com

Password *

Create Account

Or sign up using

Google Microsoft

Already have an account? [Sign In](#)

By creating an account, you agree to the [Terms of Use](#) and [Privacy Policy](#).

© 2026 OpenVPN Inc. All Rights Reserved

Help

Select the **Access Server** product and complete the account information setup process.



Select your product to continue

You can switch products at any time.

OpenVPN Self-hosted

Access Server

Host your own business VPN server.

- Deploy on Docker, Linux, Hyper-V, and VMware ESXi
- Quick launch template for AWS, Azure, GCP and all major cloud providers
- Internet traffic is routed through the installed Access Server
- Identity-based access control, device identity, and IP address checks, more coming soon
- Routing based on domain names is coming soon
- Enable high availability and load balancing across a cluster of Access Server

Select

OpenVPN Cloud-delivered

CloudConnexa®

Connect to our managed business VPN, ZTNA, and SSE service.

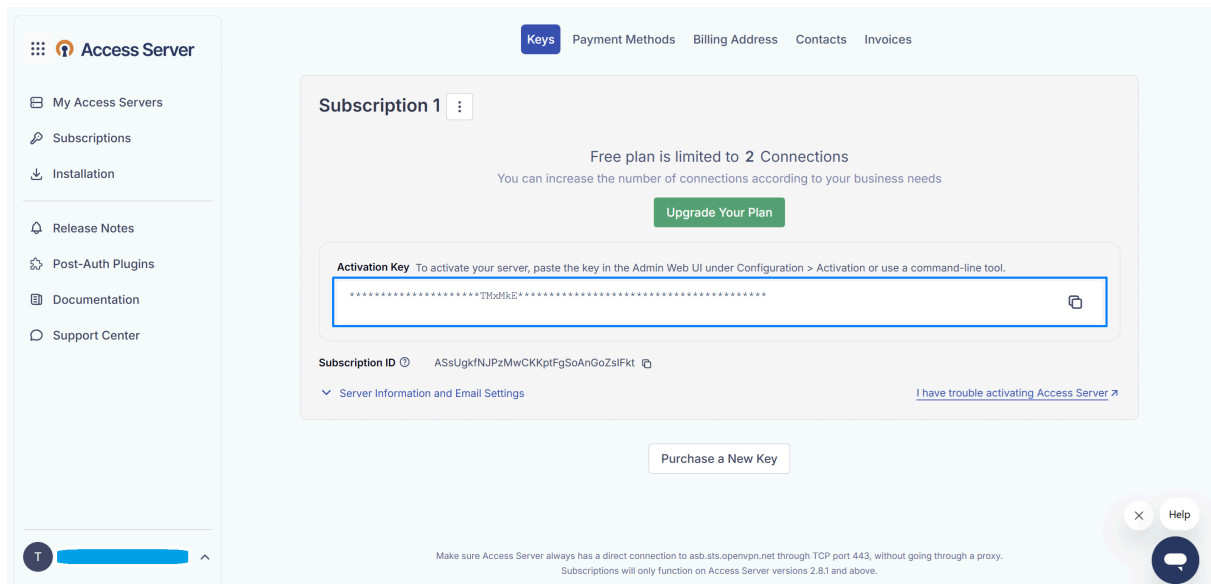
- 30+ worldwide PoPs create a private overlay network
- Connect your networks with OpenVPN or IPsec
- Need to connect an internet gateway for internet access through the VPN
- Full suite of zero trust controls
- Content filtering and IDS/IPS
- Simple routing configuration based on application domain names

Select

Compare Our Products ↓

Help

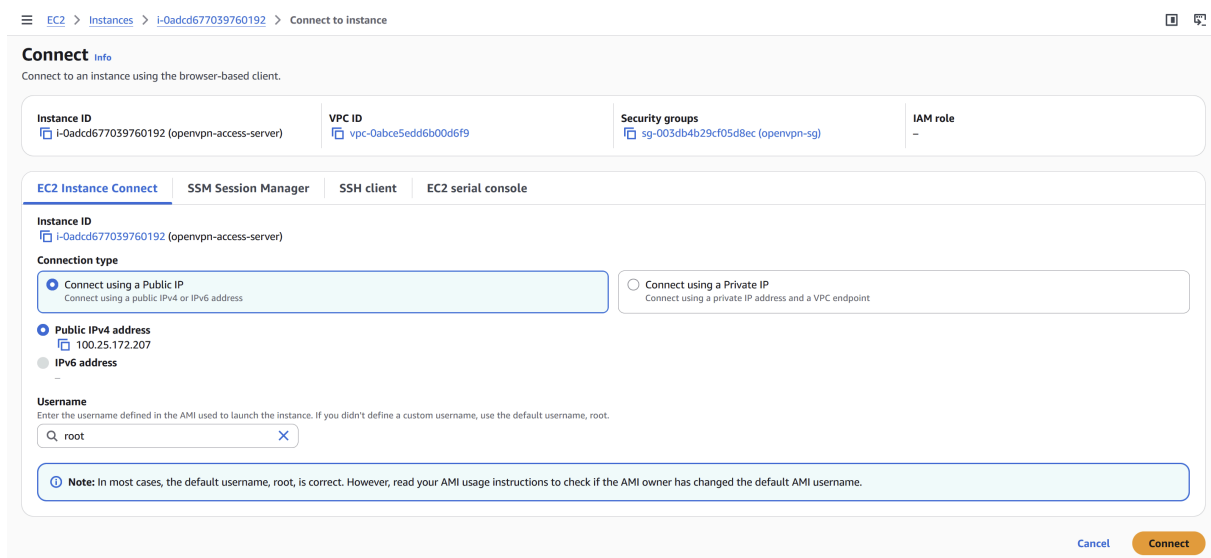
You will then be redirected to the OpenVPN application console. From the **Subscription** menu, you will see the default subscription plan set to **Free Plan**, along with the **Activation Key** used to activate your OpenVPN Access Server instance. Copy this key for use in the next step.



4.2 Initial Setup

Once the EC2 instance has finished provisioning, OpenVPN Access Server is actually already pre-installed inside the instance through the AWS Marketplace AMI. However, the system still requires an initial setup before it can operate fully.

Access the `openvpn-access-server` EC2 instance through the AWS Console.



By default, when accessing the instance for the first time, OpenVPN Access Server will prompt you to perform the initial configuration. Alternatively, you can run the following command to start or re-run the setup process:

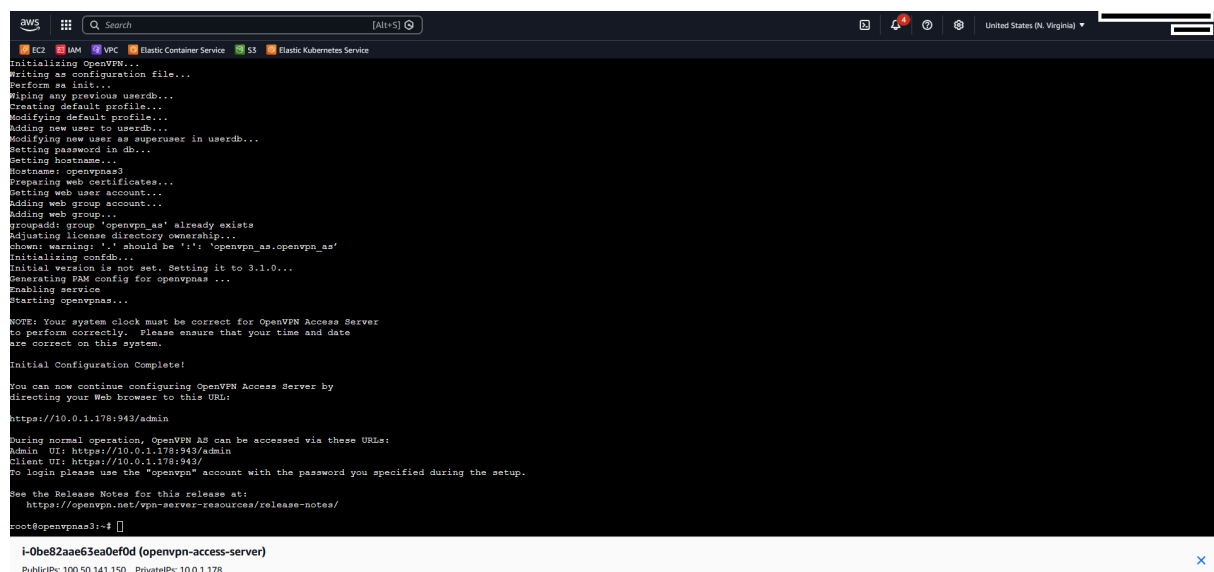
```
sudo ovpn-init
```

During the initial setup, you will be prompted to configure several settings. (You can type **Yes** or press **Enter** to use the default configuration.)

The setup process includes:

- Accepting the license agreement
- Confirming the Primary Node role
- Selecting the network interface
- Choosing the encryption algorithm
- Configuring the ports for the Web Admin UI and OpenVPN Daemon
- Setting the password for the Web Admin UI
- Entering the Activation Key collected in the previous step
- Configuring other optional settings

After the installation is completed, you will see both the **Admin UI** and **Client UI** URLs. Please note that these URLs use private IP addresses by default, so you will need to replace them with the corresponding public IP address when accessing them externally.



```
aws
Search [Alt+S]
United States (N. Virginia)

Initializing OpenVPN...
Writing as configuration file...
Perform as init...
Wiping any previous userdb...
Creating default profile...
Modifying default profile...
Adding new user to userdb...
Modifying new user as superuser in userdb...
Setting password in db...
Setting hostname...
Hostname: openvpnas3
Preparing web certificates...
Setting web user account...
Adding web group account...
Adding web group...
Groupadd: group 'openvpn_as' already exists
Adjusting license directory ownership...
shown warning: '/' should be '/'
Initializing confd...
Initial version is not set. Setting it to 3.1.0...
Generating IAM config for openvpnas ...
Enabling service
Starting openvpnas...

NOTE: Your system clock must be correct for OpenVPN Access Server
to perform correctly. Please ensure that your time and date
are correct on this system.

Initial Configuration Complete!

You can now continue configuring OpenVPN Access Server by
directing your Web browser to this URL:

https://10.0.1.178:943/admin

During normal operation, OpenVPN AS can be accessed via these URLs:
Admin UI: https://10.0.1.178:943/admin
Client UI: https://10.0.1.178:943/
To login please use the "openvpn" account with the password you specified during the setup.

See the Release Notes for this release at:
https://openvpn.net/vpn-server-resources/release-notes/

root@openvpnas3:~#
```

i-Obe82aae63ea0ef0d (openvpn-access-server)
PublicIP: 100.50.141.150 PrivateIP: 10.0.1.178

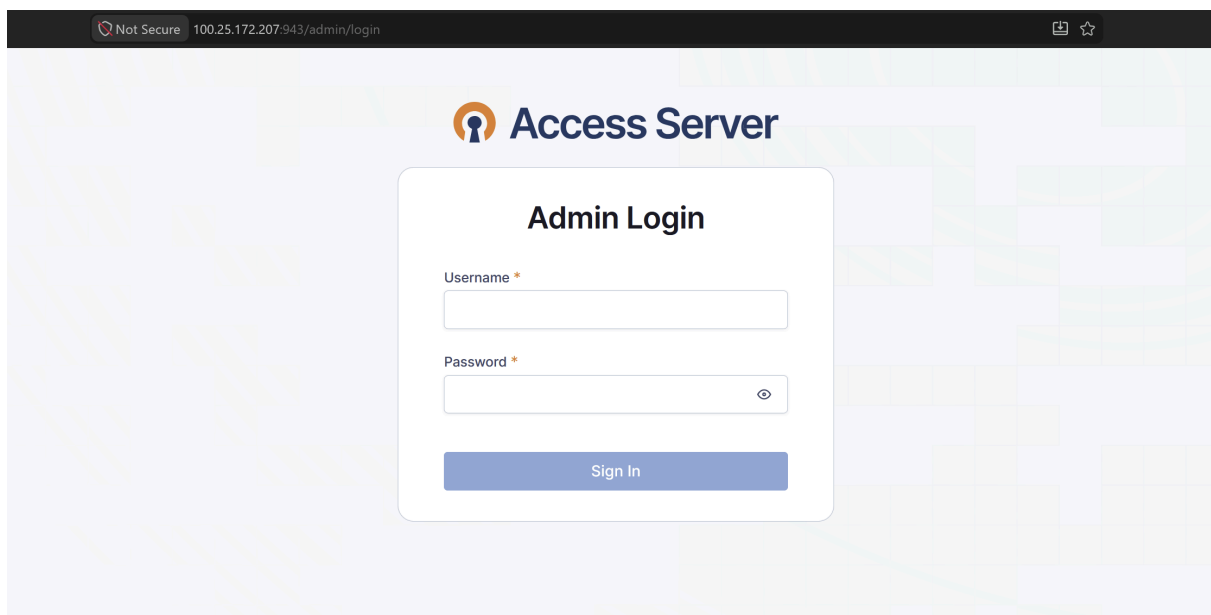
4.3 Accessing the Admin UI & Client UI

After the initial setup is completed, OpenVPN Access Server provides two interfaces:

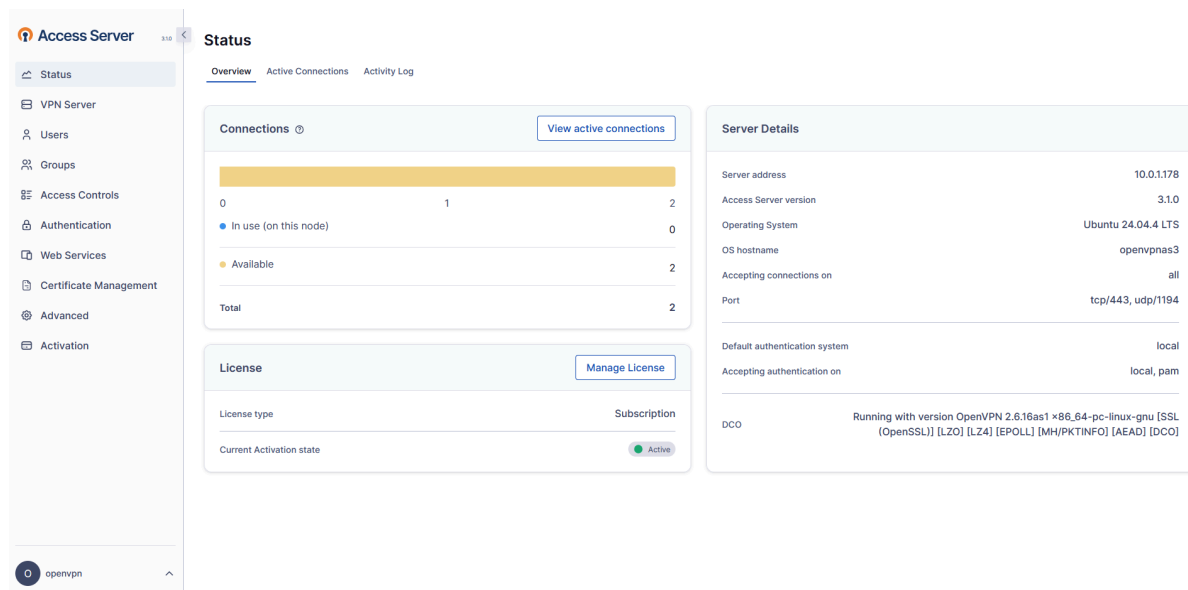
- **Admin UI:** used to manage users, routing, authentication, and VPN configurations.
- **Client UI:** used to download the OpenVPN Client application, download connection profiles, and connect to the VPN.

Access the Admin UI in your web browser using the Public IP Address of the `openvpn-access-server` EC2 instance:

```
https://<Public-IP>:943/admin
```



Log in using the username `openvpn` and the password configured in the previous step. If you skipped that step, you will be prompted to set a new password.



By default, OpenVPN Access Server is typically configured to use the EC2 instance's private IP address as the Server Address. You can verify this information under **Server Details > Server address**. However, in this lab environment, clients will connect from the public Internet. If the private IP address remains unchanged, the generated VPN profile will contain a private address, preventing external clients from connecting successfully.

Therefore, after logging into the Admin UI, we need to update the Server Address to use the Public IP Address, specifically the Elastic IP Address associated with the EC2 instance.

Navigate to the **VPN Server** menu:

- Enter the Public IP Address in the **Hostname (or IP address)** field
- Click the **Save** button
- Click the **Restart** button

Access Server 3.10

You have pending changes. Restart Access Server to apply. **Restart** 3

VPN Server 2 **Save** Cancel

Network Settings Subnets Clustering & Failover Security / Encryption Data Channel Offload Advanced

Server Address

Enter the address that VPN clients use to contact your VPN server
Use a fully qualified domain name (FQDN) where possible.

Hostname (or IP address) ①
100.50.141.150 1

Interface

Make the VPN server available on selected interfaces

Interface
All interfaces

OpenVPN daemons ②

Set the number of OpenVPN processes for incoming VPN connections on specified protocols and ports.

Protocols Protocol
TCP and UDP

OpenVPN UDP Daemon
System supports up to 2 daemons.

UDP Port ③
1194

Number of daemons ④
2

Configure TCP Daemons TCP Port ⑤ Number of daemons ⑥

openvpn

Wait for OpenVPN Access Server to restart. Afterward, you should verify that the Server Address has been updated to the public IP address.

The next step is very important. **Access Controls**, also known as **Routing Policy**, are used to route connections to private resources. In the Admin UI, create the first policy through the **Access Controls** menu:

- Click the **New Access Rule** button
- Enter the username **openvpn**
- Enter the CIDR **10.0.0.0/16**
- Click **Save rule**
- Click the **Restart** button to apply the configuration changes

Access Server 3.10

Access Controls

Group and User Access Rules Global Access Rules Intergroup Connectivity Internet Access and DNS InterClient Communication

Access Rules ① **New Access Rule**

Target Access Rule to

Group **User** 2

Username
openvpn 3

A single IP address, CIDR block or domain name ④
10.0.0.0/16 4

Protocol All Port All Reachable via NAT

5 **Save rule** Cancel

All Group User Filter

Access Rules

Define access to IP addresses, subnets, or domain names at the group or user level.

openvpn

Once the Access Controls configuration is completed, OpenVPN Access Server will begin pushing routes to VPN clients, allowing traffic destined for the VPC CIDR to be routed through the VPN tunnel into the AWS VPC. You will verify this behavior in the following steps.

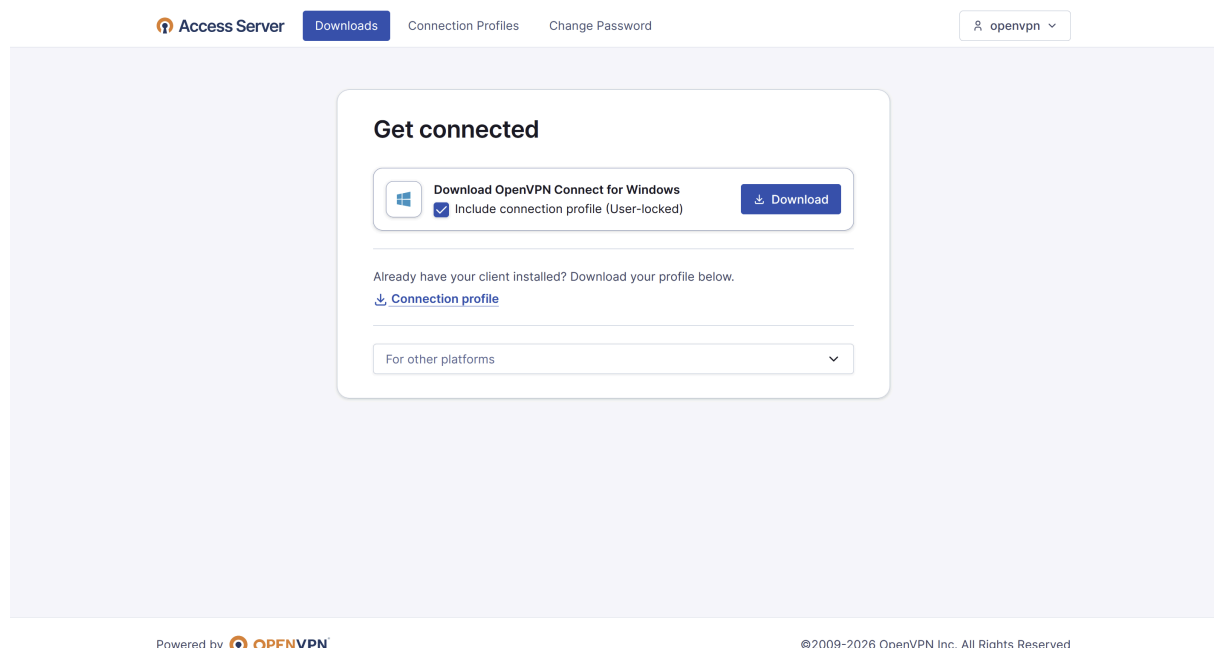
Next, open a new browser tab and access the Client UI:

```
https://<Public-IP>:943/
```

Since you have already logged into the OpenVPN Admin UI, your session credentials will also be reused for the Client UI.

In this application, you will perform the following tasks:

- Download and install the OpenVPN Client application for your computer or smartphone. Be sure to uncheck the option **Include connection profile (User-locked)** so that the connection profile can be customized later. OpenVPN clients are also available for multiple platforms including Windows, macOS, Linux, iOS, and Android.
- Download the **Connection Profile**. All required routing information, certificates, and VPN configurations are already embedded inside this profile.

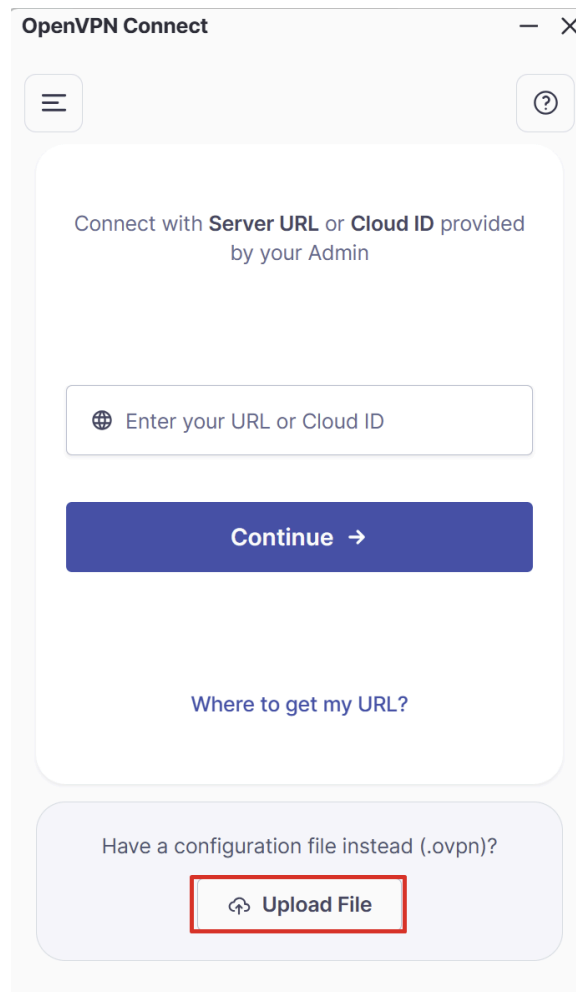


5. Configuring OpenVPN Client

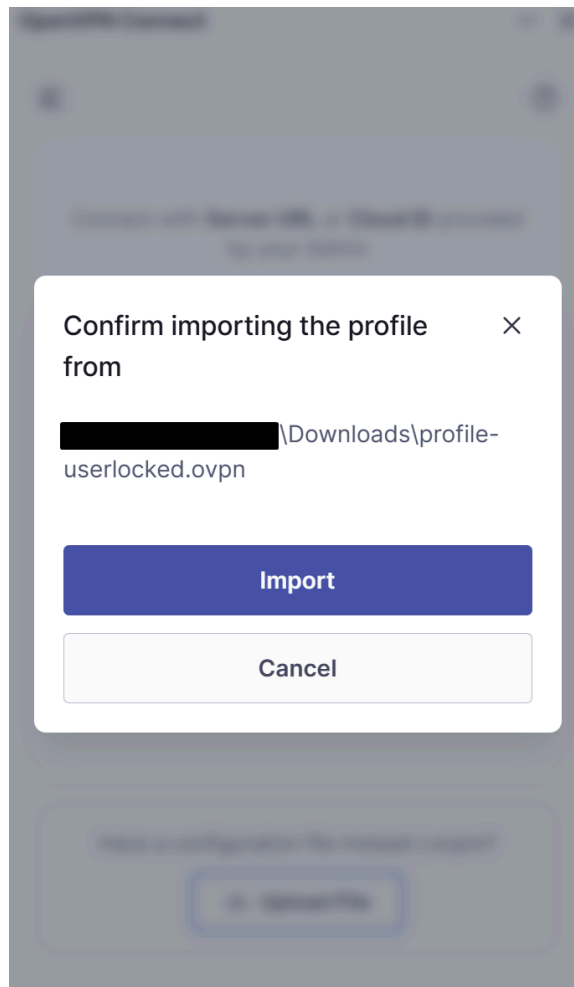
At this point, the infrastructure setup is nearly complete. OpenVPN Access Server is now running on AWS, the client profile has been downloaded, and the private EC2 instance is quietly waiting inside the Private Subnet to be accessed.

Now comes the most interesting part: turning your laptop on the public Internet into a member of the private AWS network through three simple steps:

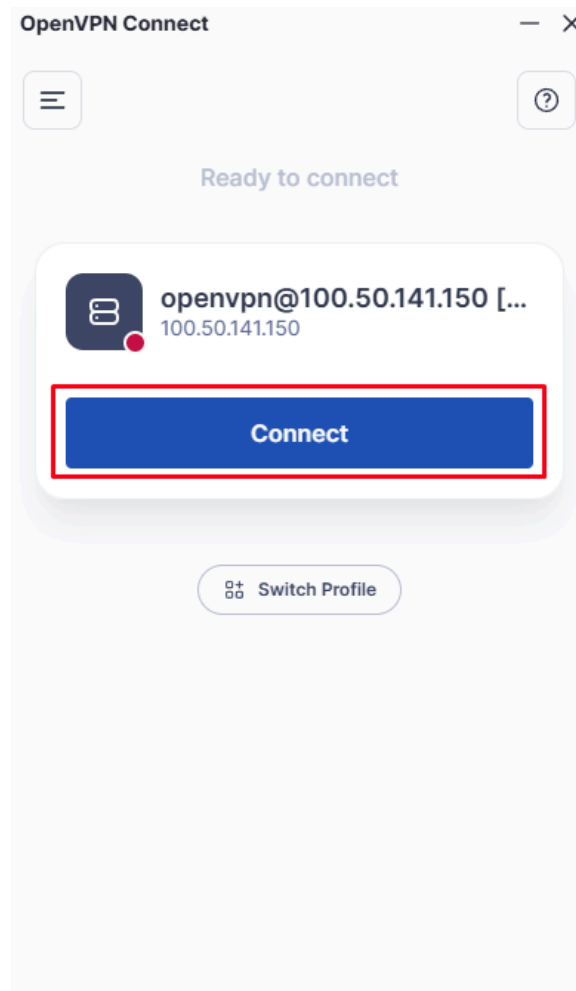
- Open the OpenVPN Client application installed earlier.



- Upload the Connection Profile file downloaded in the previous step.



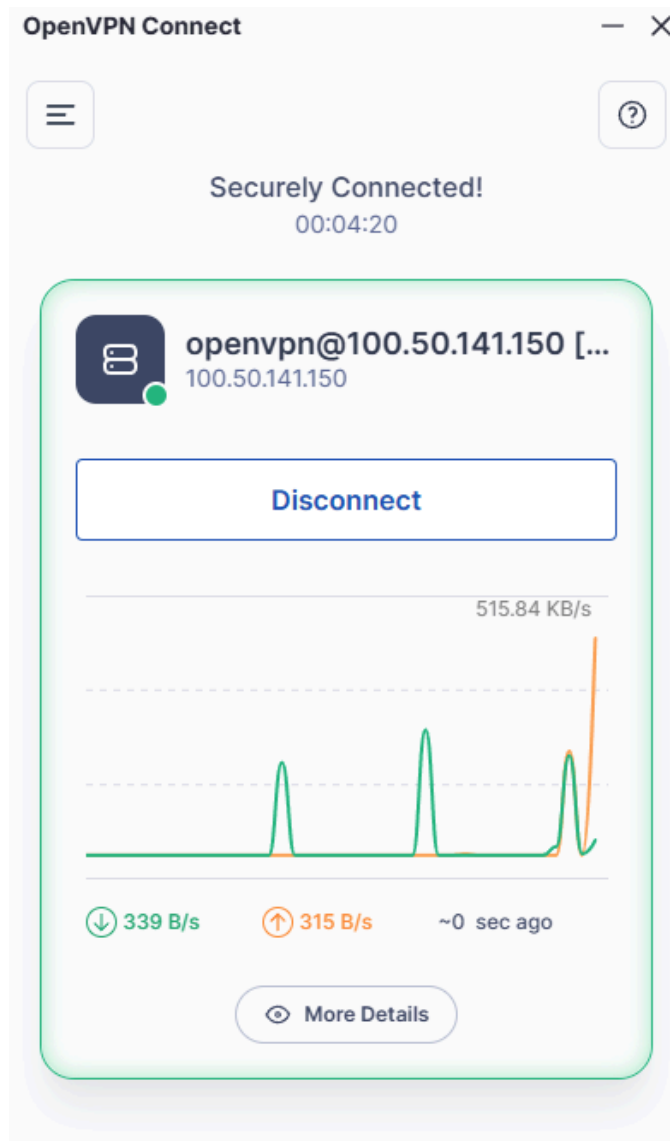
- Click **Connect** and enter your password.



The moment the **Connect** button is pressed, many things begin happening behind the scenes almost invisibly:

- OpenVPN Client starts the handshake process with the VPN Server
- Authentication is performed
- A TLS tunnel is established
- Certificates are validated
- The route table is updated
- A virtual network interface is created
- A VPN IP address is assigned to the device

The entire process usually takes only a few seconds. If everything works correctly, the OpenVPN Client status will change to **Securely Connected**, along with a small traffic graph.



After the VPN connection is successfully established, your operating system will display a new virtual network interface. You can verify the new interface using the following commands:

```
# For Windows
ipconfig /all

# For macOS
ifconfig

# For Linux (Ubuntu, Debian, RHEL, ...)
ip addr show
```

```
Command Prompt

Windows IP Configuration

Host Name . . . . . : orez-pc
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No
DNS Suffix Search List. . . . . : fpt

Unknown adapter Local Area Connection:

Connection-specific DNS Suffix . :
Description . . . . . : TAP-Windows Adapter V9 for OpenVPN Connect
Physical Address. . . . . : 00-FF-15-48-99-A9
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::3da0:5249:54bd:bd14%4(Preferred)
IPv4 Address. . . . . : 172.27.232.2(Preferred)
Subnet Mask . . . . . : 255.255.252.0
Default Gateway . . . . . :
DHCPv6 IAID . . . . . : 989921045
DHCPv6 Client DUID. . . . . : 00-01-00-01-29-BF-A7-67-04-42-1A-22-AF-13
DNS Servers . . . . . : 10.0.0.2
NetBIOS over Tcpip. . . . . : Enabled

Ethernet adapter Ethernet:
```

If you want to dig a little deeper, you can also inspect your operating system's route table. You will notice that traffic destined for the AWS VPC is now routed through the VPN interface instead of going directly to the Internet.

This mechanism that allows a private EC2 instance to remain without a Public IP address while still being securely accessible from your personal laptop.

6. Testing VPN Connectivity

From the AWS VPC's perspective, your computer has now become a member of the internal network. You now have a "secure tunnel" that allows access to resources inside the Private Subnet — in this case, establishing an SSH connection to the private instance `private-test-server`.

When provisioning the AWS infrastructure, Terraform also creates an SSH key pair to provide authentication for the private instance. Run the following command to connect:

```
ssh -i vpn-demo-ssh-key.pem ec2-user@<Private-Instance-IP>
```


VPNs are far more than tools for masking IP addresses. It is also one of the most important solutions for protecting infrastructure and controlling access in cloud environments. Through this article, we explored the fundamental concepts of VPNs, how VPNs work, the problems they help solve, and successfully deployed OpenVPN Access Server on AWS to securely access private resources.

Although VPNs significantly improve security, the VPN Server itself is also a critical access point that must be carefully protected. Some recommended security best practices include:

- Use strong passwords and avoid using default accounts
- Enable MFA whenever possible
- Restrict Security Groups following the principle of least privilege
- Rotate credentials and certificates regularly
- Avoid directly exposing internal resources to the public Internet
- Monitor access logs to detect suspicious activities

Understanding VPN fundamentals is essential for building secure cloud infrastructure. OpenVPN provides a practical and accessible way to implement secure remote access while keeping critical systems private.

References

- OpenVPN Documentation (<https://openvpn.net/as-docs/>)
- AWS VPC Documentation (https://docs.aws.amazon.com/vpc/?utm_source=chatgpt.com)
- AWS EC2 Documentation (https://docs.aws.amazon.com/ec2/?utm_source=chatgpt.com)